A Technical Account of a Failure-Managed Frontier Explorer: Algorithm, History, and Evidence

Author A Author B

Affiliation withheld

Internal technical report. Companion to the concise manuscript.

Source of record: `main` at `de8209a`.
Explorer file last changed in `eccbb97` (1,448 lines).

Abstract

This report is the source-level description of the frontier explorer in `Ros_lidar_bot`, the navigation and safety configuration that constrains it, and the commit history that produced the current policy. The explorer is not a new information-gain planner. It is a full-grid classical frontier detector whose utility is a weighted sum of cluster size, Euclidean distance, heading change, and a local free-space fraction, surrounded by a failure policy. The policy deletes unknown cells that a border-seeded flood fill cannot reach, remembers those cells as an exact set, counts Nav2 successes that leave the frontier in place, escalates repeated aborts, and keeps a geographically separated queue that a safety-stop Boolean can preempt. The report records every configured parameter and every hard-coded constant that changes a decision, the order of filters, the state of a goal, and the places where the June 2026 reference note no longer matches the code. It also records what the repository does not contain: rosbags, coverage logs, baselines, and ablations. No performance number in this document comes from a robot trial. Arithmetic examples are labelled as such. A separate assessment treats conference readiness and intellectual property. This document is the one a later experiment or a later revision should treat as the description of the implementation.

Keywords: frontier exploration, occupancy grid, blacklist, phantom success, Nav2, safety stop, reproducibility.

Contents

1 How to read this report **4**

2 Platform **4**

2.1 What the explorer is allowed to see 4

2.2 Command path 4

2.3 Safety stop 5

2.4 Navigation parameters that change exploration outcomes 6

3 Problem formulation **7**

3.1 Grid and pose 7

3.2 Frontier . . . 7
3.3 Decision . . . 7
3.4 Research question, stated as a question . . . 7

4 Related work and the novelty boundary **8**

5 Detection, pruning, and score **9**
5.1 Enclosed unknown space . . . 9
5.2 Prune order . . . 9
5.3 Utility . . . 10
5.4 Queue construction . . . 11
5.5 Pop score . . . 11

6 Execution **12**
6.1 Timer . . . 12
6.2 In-progress checks, in order . . . 13
6.3 Goal generation . . . 13
6.4 Pseudocode . . . 13

7 Blacklists **15**
7.1 Ordinary entries . . . 15
7.2 Aborts and phantom successes . . . 15
7.3 Manual zones . . . 15
7.4 State machine . . . 16

8 Calibration and relaxation **17**

9 Worked arithmetic **18**

10 Parameters **19**

11 Complexity **21**

12 History **22**

13 Where the June report is wrong **24**

14 Evidence **25**

15 What an evaluation has to be **26**

16 Threats to validity **27**

17 Reproducibility **28**

18 Publication and intellectual property, in brief **28**

19 Conclusion **28**

1 How to read this report

The concise manuscript argues the research question in conference form. This report is the authority for behaviour. When the two disagree, this report and the source win, in that order: the source wins over both papers.

Three kinds of statement are marked as they occur.

- **Source.** A behaviour that follows from the current file, with a line range in `frontier_explorer_node.py` unless another file is named.
- **Record.** A motivation taken from a commit message or from a comment introduced by that commit. A record is evidence of what the author wrote, not evidence that the change improved exploration.
- **Gap.** A claim that would require a measurement the repository does not have.

Simulation worlds named in old commits (`office`, `warehouse`, `room`) are not in the `main` tree. They remain on the `Simulation` branch history. Hardware launch files are on `main`. Nothing in this study was executed on a robot or in Gazebo by the author of this report.

2 Platform

2.1 What the explorer is allowed to see

The node subscribes to `/map` (`OccupancyGrid`), `/safety_blocked` (`Bool`), and `/manual_blacklist` (`String`). It looks up `map→base_footprint`. It publishes `/frontier_debug` (`MarkerArray`) and action goals on `navigate_to_pose`. It does not subscribe to `/scan`, `/cmd_vel`, the camera, or the YOLO topics. Semantic markers cannot change a frontier score. That is a scope limit, not an oversight discovered in a comment: the subscriptions are the complete list (lines 155–161).

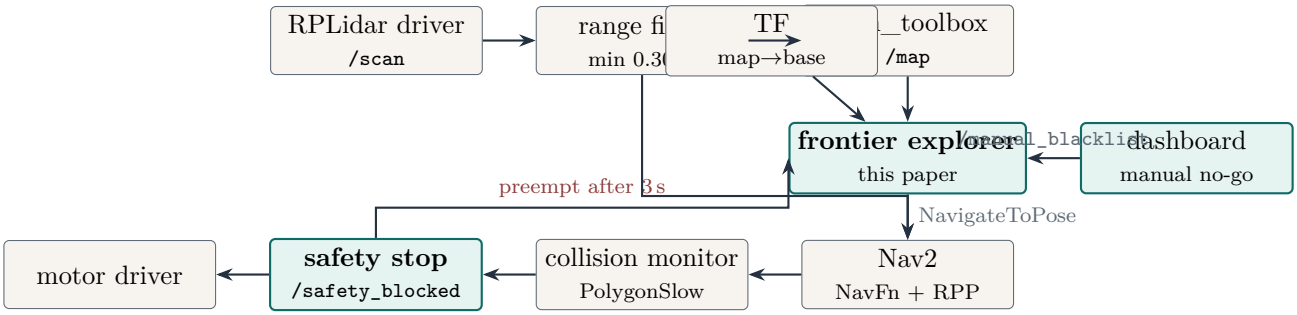


Figure 1: Process boundary. Teal nodes are in this repository. Arrows are the topics and the action that the explorer actually uses. The camera pipeline is absent because it is absent from the node.

2.2 Command path

`config/nav2_params.yaml` documents the Jazzy chain: regulated pure pursuit writes a navigation command, a velocity smoother follows, the collision monitor follows, and only then does `/cmd_vel` reach the safety stop, which publishes `/cmd_vel_safe` for the driver. Launch comments in `launch_robot.launch.py` still say Nav2 publishes `/cmd_vel` directly into the safety stop. The

parameter file is the wiring that `navigation_launch.py` is given. This report follows the parameter file. The chain was not observed at runtime.

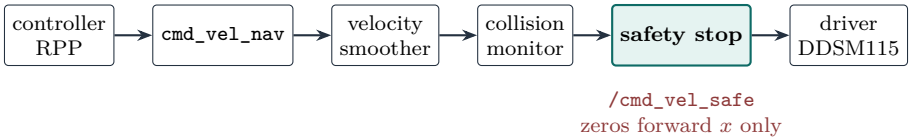


Figure 2: Command path as configured, not as re-measured. PolygonSlow is enabled. PolygonStop is present and disabled. Regulated pure pursuit collision checking is off.

The collision polygon that is enabled is a slowdown box from $x = 0.30$ m to 0.65 m and $|y| \leq 0.24$ m, with `slowdown_ratio` 0.5. The stop polygon from 0.30 m to 0.40 m is disabled. The safety stop is therefore the hard gate. It is not the first gate: a person inside the slowdown box can already have halved the command.

2.3 Safety stop

`safety_stop_node.py` and the parameters in `launch_robot.launch.py` (lines 222–236) agree. The values below are the launched values, not the defaults that would apply if the node were started alone. The defaults happen to match.

Table 1: Safety-stop parameters that the explorer can observe, directly or through a blocked robot.

Name	Value	Unit	Effect on exploration
`min_safe_distance`	0.35	m	Forward block engages
`clear_margin`	0.10	m	Release only beyond 0.45 m
`ignore_below`	0.30	m	Chassis returns ignored
`front_opening_deg`	50	deg	Total width, so ±25°
`rear_opening_deg`	50	deg	Reverse only; does not set the Boolean
`odom_raw_timeout_sec`	0.5	s	Silence zeroes all motion and sets the Boolean

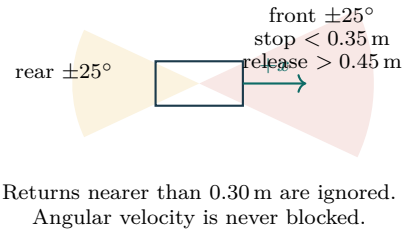


Figure 3: Laser sectors. Only the forward sector, or stale `/odom_raw`, raises `/safety_blocked`. Yaw rate is copied through unchanged, so a spin recovery still turns the robot while translation is zero.

The Boolean is published from the scan callback when the forward state is recomputed, and from the 10 Hz watchdog when odometry is stale. It is not a fixed 20 Hz stream. The June report's "20 Hz" and its 0.25 m, 180° cone are the old simulation launch, not this bringup. The cone was narrowed on 26 July 2026 (`91d464f`). The commit message says a ±90° cone froze the robot when a person stood beside it. That is a record of intent. There is no count of freeze events.

The explorer's own timer is 3 s. It stores the time of the rising edge and preempts when the Boolean has stayed true for more than 3 s at a tick (lines 290–318). The effective delay is therefore

the threshold plus up to one timer period. A rear-only block never starts that clock. An odometry dropout does, and the goal is then blacklisted even though the map may be fine.

2.4 Navigation parameters that change exploration outcomes

These are not explorer parameters. They decide whether a goal the explorer sends is reached, aborted, or stalled.

Table 2: Nav2 settings that interact with the explorer. Source: `config/nav2_params.yaml`.

Setting	Value	Interaction
Planner	NavFn, A*	`allow_unknown: true`, tolerance 0.5 m. Goals lie in free cells, but the path may enter unknown space.
`xy_goal_tolerance`	0.30 m	Nav2 can succeed before the explorer's 0.45 m bookkeeping radius.
`yaw_goal_tolerance`	0.50 rad	Orientation is the bearing from the robot to the goal at send time. It is not tracked afterwards.
Pose progress checker	0.08 m or 0.30 rad within 25 s	Rotation counts. The explorer's stall test does not.
`desired_linear_vel`	0.35 m/s	Kinematic lower bound only. A 5.5 m goal takes at least $5.5/0.35 \approx 16$ s of translation, ignoring turns and the slowdown polygon.
`use_rotate_to_heading`	true	Turn in place above 0.4 rad, yaw rate 1.0 rad/s. Re-enabled 20 July after a dead-odometry spin was charged to this flag. The safety stop now zeroes motion if `/odom_raw` dies.
`use_collision_detection`	false	RPP does not stop on its own lookahead.
Local inflation	0.28 m	Footprint is a 0.50×0.36 m rectangle.
Global inflation	0.35 m	Lowered from 0.45 m so one person does not seal a corridor. Record, not a trial.
`transform_tolerance`	1.0 s	Several servers. Comment says map→odom lagged about 0.6 s on the Pi under DenseBoost.
Behaviour plugins	spin, backup, wait	No behaviour tree file is in the repository. Recovery order is Nav2's default for this plugin list.

SLAM Toolbox runs in online async mode. Resolution is 0.05 m. Minimum laser range in the SLAM file is 0.45 m. The scan filter that actually edits `/scan` uses 0.30 m (`launch_robot.launch.py`). Those two floors differ. The explorer never reads the scan, so the difference matters only through the map and through the safety stop, which sees the filtered scan and then ignores returns below 0.30 m again.

Startup order is SLAM at 5 s, Nav2 at 8 s, explorer at 22 s (`autonomous_robot.launch.py`, lines 61–65). The 22 s delay is a fixed timer, not a wait on the action server. The server wait of 1 s happens later, inside each goal send.

3 Problem formulation

3.1 Grid and pose

Let M be the latest occupancy grid, width W, height H, resolution ρ, origin (o_x, o_y). Cell (i, j) has world centre

$$(o_x + (i + \tfrac{1}{2})\rho,\ o_y + (j + \tfrac{1}{2})\rho). \tag{1}$$

The implementation classifies a cell as free only when its integer value is 0 and unknown only when it is -1 (lines 960–961). Occupied cells are typically 100 under SLAM Toolbox's trinary maps. Any other value, including a probabilistic midpoint, is neither free nor unknown: it blocks the flood fill and it is not a frontier neighbour. The report does not claim the live map is trinary. The mapper file does not set a map mode.

The pose $p = (x, y, \psi)$ is the translation and yaw of `base_footprint` in `map`. Yaw is the standard z-w extraction (lines 1131–1134). If the transform is missing, the tick returns. No goal is sent and the empty-tick counter is not incremented, because the increment sits after a successful pose lookup on the idle path (lines 382–408).

3.2 Frontier

An unknown cell is a frontier cell when at least one of its eight neighbours is free. The four roll-shifts that build that test wrap around the array, so the code then clears the entire border of the mask (lines 1022–1038). Frontier cells on the outer row and column are never detected, regardless of the later border margin.

Clusters are the 8-connected components of that set (lines 1054–1071). The representative point $g(F)$ is not the centroid. The centroid index is the rounded mean of member indices. Among free 8-neighbours of any member, g is the one with smallest squared index distance to that centroid, placed at the cell centre (1) (lines 1073–1103). If no free neighbour exists, the cluster is dropped. The robot is asked to drive to known free space beside the unknown set. NavFn may still plan through unknown cells.

3.3 Decision

On each idle tick the policy outputs either a pose or nothing. The pose orientation is the bearing from the robot to g at send time (lines 520–530). It is not updated as the robot moves. During a goal the policy may cancel, blacklist, and replace the pose. It has no terminal state. "Exploration complete" is a log emitted when the empty counter reaches its patience (lines 411–414). The timer continues, relaxation continues, and a later candidate is still sent.

3.4 Research question, stated as a question

The implementation embodies this question and does not answer it.

> On a classical frontier detector feeding an unmodified Nav2 stack, do an exact-cell rejection of enclosed unknown space, a counter of successes that leave unknown cells in place, and a safety-triggered standby queue reduce repeated goals in the same map region, relative to nearest-frontier and to a gain-over-path-cost utility, without a compensating loss of coverage?

Section 15 turns the question into measurements. Section 14 states the present answer: not known.

4 Related work and the novelty boundary

Nearest-frontier exploration is Yamauchi's rule [1, 2]. Utility exploration replaces distance with a trade of expected information against a cost [3, 4, 5]. Holz, Basilico, Amigoni, and Behnke showed that the ranking of those strategies is environment-dependent and that the cost model can dominate the gain model [6]. Fast detectors avoid a full-grid scan [7, 8]. Sampling-based detectors avoid an explicit boundary [9]. `explore_lite` clusters frontiers, scores a gain against a path cost, and blacklists failures [10].

Table 3 places the implementation against that set. "Present" means the source does it. "Absent" means it does not. Neither column is a quality judgment.

Table 3: What is and is not in the explorer, relative to standard practice.

Idea	In this source?	Note
Unknown cell beside free space	Present	8-connected. Yamauchi used the free-side boundary; the goal here is moved back onto free space.
Full-grid extraction each decision	Present	Not WFD, not RRT.
Linear utility of size, distance, heading, openness	Present	Weights are configured, not learned.
Path cost or information gain	Absent	Distance is Euclidean. Size stands in for gain.
Blacklist of failed goals	Present	Also in `explore_lite`. Radius and timeout adapt with hits.
Enclosed-void removal	Present	4-connected flood from the map border. The predicate is an exact cell set, not a disk.
Success that does not clear the map	Present	Counted separately from abort. Threshold 2.
External safety Boolean as a goal switch	Present	3 s, then the queue.
Multi-robot, semantic goals, next-best-view raycast	Absent	Camera stack is parallel and unused.

The combination in the last three "present" rows is the only part that should be described as the authors' packaging. It is not, on the evidence in this repository, a demonstrated improvement, and it is not a licence to claim the surrounding classical method. A prior-art search is still required before any of those three rows is called new. Flood-fill of unreachable cells and goal blacklists are old. The exact-cell encoding and the phantom counter are specific; specificity is not novelty.

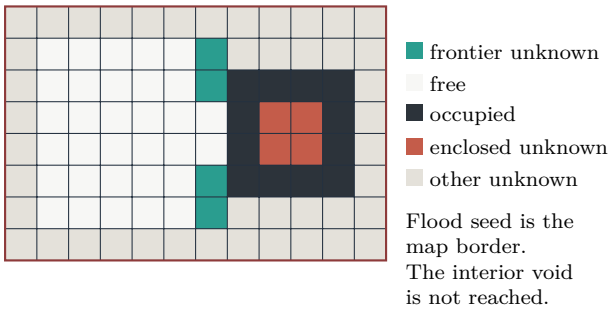


Figure 4: Schematic grid. Red cells are unknown and not 4-connected to the border through free or unknown cells. They are rewritten to occupied in a copy. Teal cells can seed a frontier. This is not a recorded map.

5 Detection, pruning, and score

5.1 Enclosed unknown space

Define $C = \{cells = 0\}$ and $U = \{cells = -1\}$. The fill (lines 899–930) is a 4-connected BFS on $C \cup U$, seeded from every traversable border cell. Enclosed cells are U minus the reachable set. If any exist, the grid copy sets them to 100 before frontier extraction (lines 963–976). Frontiers are then computed on that copy.

Each component with at least three cells may become a permanent zone (lines 991–1015):

- the centroid in world coordinates is inserted if no existing centroid lies inside a squared radius $(2r_b)^2$, where r_b is `goal_blacklist_radius` (0.80 m, so 1.60 m);
- every cell of the component, plus a one-cell Chebyshev buffer, is stored as the integer key $(\text{round}(x/\rho), \text{round}(y/\rho))$.

Components of one or two cells are still removed from frontier extraction. They simply get no permanent entry. The SLAM map is never written. The zones accumulate for the process lifetime and are not pruned when the grid origin shifts. The comment in the source says world keys remain valid across origin shifts. That is true only while ρ is constant and the rounding function is the same at insertion and at query. Both are true today. Alias is a separate problem, worked in Section 9.

The marker for a permanent zone is a cylinder of radius 1.60 m (lines 1325–1342). The predicate is the cell set (lines 1231–1238). An operator looking at RViz sees a larger exclusion than the test applies.

Connectivity does not match. The fill is 4-connected. Frontier adjacency is 8-connected. A diagonal corridor of unknown cells can be classified as enclosed and, were it not rewritten, could also have been a frontier. The rewrite happens first, so the diagonal corridor disappears. Whether that is desirable in real furniture is a **gap**.

5.2 Prune order

Figure 5 is the order inside `_score_frontiers` (lines 750–793). Counters `n_small`, `n_no_target`, `n_edge`, `n_blacklist`, and `n_dist` are debug logs only. The openness rejection increments `n_edge`, so the log line conflates border rejects with wedge rejects.

The border gate is

$$m = \max\left(1,\ \text{int}(d_{\text{wall}}/\rho)\right), \tag{2}$$

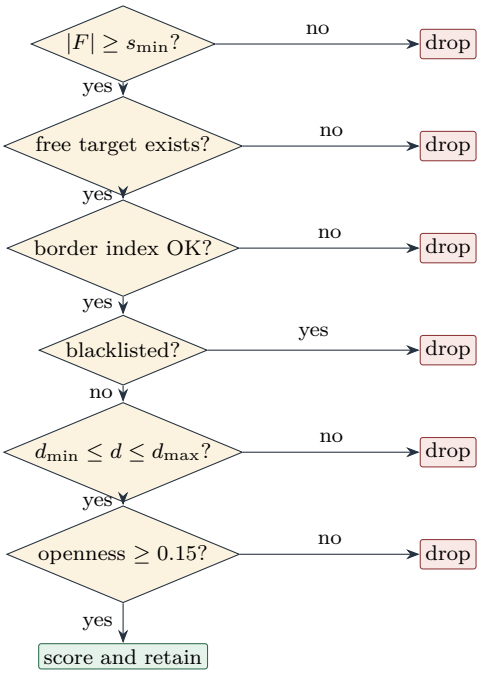


Figure 5: A cluster is scored only if it passes every test. The openness floor and the border test are hard gates, not soft penalties.

and the goal's cell index must lie in $[m,\, W - m) \times [m,\, H - m)$. With the launched $d_{\text{wall}} = 0.15\,\text{m}$ and $\rho = 0.05\,\text{m}$, $0.15/0.05$ is the float $2.999\ldots$, and Python `int` truncates toward zero, so $m = 2$, not the 3 written in the YAML comment and in the June report. The accepted cell nearest the border has its centre $2.5\rho = 0.125\,\text{m}$ from the map edge, which is inside the $0.15\,\text{m}$ the parameter name suggests. The test is an index margin, not a Euclidean distance to the boundary.

Openness uses a square window of radius 3 cells, not a disk (lines 1105–1128):

$$\Omega(g) = \frac{\#\{\textit{cells in } [-3,3]^2 \text{ with value } 0\}}{\#\{\textit{cells of that window inside the grid}\}}. \tag{3}$$

The floor is $\Omega \geq 0.15$, hard-coded. On 13 May the previous floor of 0.25 was removed (`82dbf87`). The message says it rejected all tight frontiers in warehouse and office worlds. On 3 June a floor of 0.15 was put back (`6b7e6c2`) with a comment that it should catch only solid-furniture wedges. No rejection counts are stored for either change.

If the pass at $D = d_{\max}$ returns no goal, the same clusters are scored once more at $D = \min(2d_{\max},\, d_{\text{cap}})$ with $d_{\text{cap}} = 12\,\text{m}$ (lines 709–718). Expansion is one step, not a loop. Both passes rebuild the queue.

5.3 Utility

Let $S = \max |F|$ over the clusters of this call, and let D be the distance limit of this pass. For robot pose p and yaw ψ,

$$\begin{aligned} \sigma_{\text{size}} &= |F|/S, & \sigma_{\text{dist}} &= \|g - p\|/D, \\ \sigma_{\text{head}} &= |\text{wrap}(\text{atan2}(g_y - y, g_x - x) - \psi)|/\pi, \\ \sigma_{\text{open}} &= \Omega(g). \end{aligned} \tag{4}$$

$$J(F) = w_s\sigma_{\text{size}} - w_d\sigma_{\text{dist}} - w_h\sigma_{\text{head}} + w_o\sigma_{\text{open}}. \tag{5}$$

Launched weights are $w_s = 1.8$, $w_d = 1.0$, $w_h = 0.5$, $w_o = 0.7$. The node's `declare_parameter` defaults differ (1.5, 1.0, 0.8, 0.4) and apply only if the YAML file is not loaded. `autonomous_robot.launch.py` loads the YAML.

If every survivor has $\|g - p\| > 0.35D$, each distance term is replaced before J is recomputed:

$$\sigma_{\text{dist}} \leftarrow \max\left(0,\ \|g - p\|/D - \beta\right), \quad \beta = \min(0.40,\ 0.45\,|F|/S). \tag{6}$$

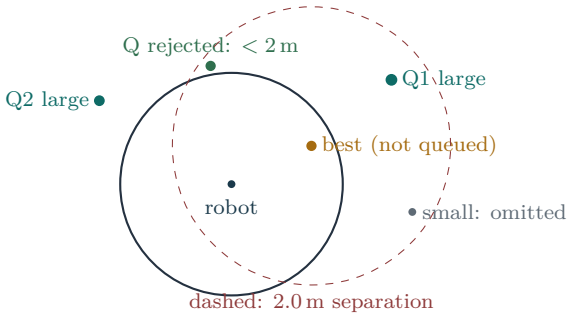
The constants 0.35, 0.40, and 0.45 are hard-coded (lines 816–845). A near survivor, even a poor one, suppresses the bonus for everyone. The bonus cannot create a candidate that pruning already removed.

5.4 Queue construction

When `collect_all` is true, which it always is in the two call sites, survivors are sorted by J. The best is the return value and is not queued. From the remainder, up to $n_q = 5$ points are kept if $|F| \geq n_L$ and each is at least $2.5r_b$ from the others. At $r_b = 0.80\,\text{m}$ that separation is 2.0 m. If none qualify, the size test is skipped and any remainder may fill the five slots (lines 854–888). n_L is set once, on the first map:

$$n_L = \max(3s_{\min},\ 15,\ \lfloor 0.0015\,WH \rfloor), \tag{7}$$

with $s_{\min}$ the minimum cluster size at calibration time, which is the configured 3 unless something else changed it first. Nothing else runs before the first map callback. Later relaxation of $s_{\min}$ does not recompute n_L.



Conceptual geometry. The best cluster is returned as the immediate goal and is not stored in the queue. The idle loop later pops the queue before it rescans.

Figure 6: Queue geometry at the configured blacklist radius. The best cluster is withheld from the queue because the full scan returns it directly. The idle loop does not use that return value when the queue is non-empty. See Section 6.

The queue stores $(g, J, |F|, \|g - p\|)$ only while sorting. The retained list stores (g, J). On pop, J is ignored.

5.5 Pop score

`_pop_queued_frontier` (lines 463–510) discards a queue older than 20 s. Otherwise it pops from the front and skips a point that is blacklisted, whose 0.8 m neighbourhood contains no unknown cell, whose distance from the robot is outside $[d_{\min},\ d_{\text{cap}}]$, or whose distance from the last blacklisted

point is below $2r_b$. The distance ceiling is the cap, 12 m, not $d_{\max}$. A point admitted when the search radius was 5.5 m can be dispatched later from 10 m away.

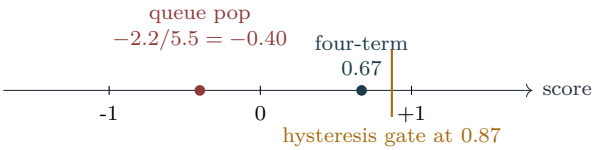
The score returned with a valid pop is not J. It is

$$J_{\text{pop}} = -\,\|g - p\|/d_{\max}. \tag{8}$$

$d_{\max}$ may already have been relaxed to 12 m, which changes J_{pop} even for the same point. The hysteresis test on the idle path (lines 423–427) is

$$\text{send only if } g_{\text{active}} = \varnothing \text{ or } J \geq J_{\text{active}} + 0.20. \tag{9}$$

It compares J_{pop} with a previously stored J whenever the previous goal position has not been cleared. Those quantities do not share a zero or a scale. Figure 7 uses the numbers from the worked example in Section 9. During an active goal the test is not used; preemption sends the popped goal unconditionally.



Same candidate can be logged as 0.67 when it is scored
and as −0.40 when it is later popped. The gate compares them.

Figure 7: Scale break between (5) and (8). The hysteresis margin of 0.20 assumes the left-hand and right-hand scores are the same function.

6 Execution

6.1 Timer

`goal_interval_sec` (3.0 s) is the period of `_tick`. It is not stored on the object. Every timeout below is sampled on this period.

The tick has two halves. If `goal_in_progress` is true, the node may cancel or replace the goal and then returns. It does not select a fresh goal in the fall-through sense: after a failed attempt to pop a replacement it still returns, because the refresh-and-return at lines 359–375 is inside the in-progress branch and is reached whenever the earlier branches do not return. There is one exception of structure. The timeout branch sets `goal_in_progress` false and, if no replacement is sent, does not return inside that branch. Control then falls through the progress `elif` (skipped, because the `if` was taken) into the queue refresh, and then returns. The next period takes the idle half, subject to the cooldown.

If no goal is in progress, a cooldown of `goal_cooldown_sec` (3.0 s) measured from `last_goal_end_time` causes an immediate return. That return does not increment the empty counter. After cooldown, the node tries the queue first and a full scan only if the pop returns nothing (lines 396–405). This is the behaviour introduced as “Q1 used in normal flow” (`818e294`) and extended so that a vanished frontier and a timeout also try the queue (`b79ad0d`). The consequence is that a queue younger than 20 s suppresses a rescan. The next goal is a previous runner-up that survives revalidation, not the current best frontier.

6.2 In-progress checks, in order

The order is the order in the source, and it matters.

1. If the active point's 0.8 m neighbourhood contains no unknown cell, set the clean-cancel flag, cancel, and try the queue. Do not blacklist (lines 261–288). The radius 0.8 m is hard-coded, not r_b. At $\rho = 0.05$ m, $\text{int}(0.8/0.05) = 16$ cells.
2. If the safety Boolean has been true longer than 3 s and the queue yields a point, blacklist the active point, cancel, and send (lines 290–318). If the queue is empty or the pose is missing, this branch does not return. The safety clock is not cleared unless a switch happens.
3. If the goal is older than 90 s, blacklist, cancel, and try the queue (lines 320–336).
4. Else if `distance_remaining` has not fallen by more than 0.12 m for 25 s, blacklist, cancel, and try the queue (lines 337–357).
5. If the queue stamp is older than 20 s, run a full scan solely to rebuild the queue (lines 359–374). The returned best goal is discarded. The active goal is left alone.

`_feedback_cb` (lines 664–674) updates the progress clock only when the remaining distance decreases by more than `min_progress_delta`. An increase, which a longer replan produces, does not reset the clock and does not replace the stored remaining distance. The next decrease is measured against the old, smaller value, so a single later sample can look like a large improvement. Until that sample, the stall clock runs. Nav2's own checker can be satisfied by a rotation of 0.30 rad. The explorer will still declare a stall. Both windows are 25 s. The YAML comment says the explorer window must be strictly longer than Nav2's `movement_time_allowance`. The configured values are equal, and the comment also mentions 12 s in the same entry. The 12 s figure is stale.

6.3 Goal generation

Each send increments `_goal_gen` and captures it in the accept, result, and feedback callbacks (lines 539–560, introduced in `badd582`). A callback whose generation is not current returns immediately. It does not clear `_clean_cancel`. The flag is cleared only at the bottom of a result callback that passed the generation test (line 662).

Sequence that leaves the flag set: a clean cancel sends a replacement, which increments the generation; the old result arrives and is ignored; `_clean_cancel` stays true until the replacement's result callback. On that later callback, a `STATUS_CANCELLED` takes the clean-cancel log and does not blacklist. Blacklisting for a safety or timeout cancel is done before `cancel_goal_async`, so the stale flag changes the log, not the exclusion. A path that expected the callback itself to blacklist would be wrong. After `47a164d` the callback never blacklists on cancel.

`goal_handle` is assigned in the accept callback. A safety preemption at 3 s can run before that assignment if acceptance is late. The source calls `self.goal_handle.cancel_goal_async()` without a null test (line 311). The initial value is `None`. This is a real crash path in the source. This report has no log showing it has occurred.

6.4 Pseudocode

The listings follow the source, including the queue-first idle path and the pop score. They omit marker drawing.

Listing 1: Idle and in-progress tick, abridged only by marker calls.

```
every 3.0 s:
  if no map: return
  if goal in progress:
    if unknown cells gone within 0.8 m of the goal:
      clean_cancel = True
      cancel()
      in_progress = False
      if queue pop succeeds: send(pop); return
      stamp cooldown; return
    if safety blocked for > 3 s and queue pop succeeds:
      blacklist(active); cancel(); send(pop); return
    if age > 90 s:
      blacklist(active); cancel()
      if queue pop succeeds: send(pop); return
    else if no 0.12 m drop in distance_remaining for 25 s:
      blacklist(active); cancel()
      if queue pop succeeds: send(pop); return
    if queue older than 20 s: rescan()  # queue side effect only
    return
  if within 3.0 s of last goal end: return
  if no pose: return
  prune expired blacklists
  if active goal is within 0.45 m: forget its position
  (g, J) = pop queue          # J is -distance/d_max
  if g is empty: (g, J) = full scan   # J is equation (score)
  if g is empty:
    empty_ticks += 1
    relax()
    if empty_ticks >= patience: log "complete"
    return
  empty_ticks = 0
  if active position exists and J < active_J + 0.20: return
  send(g, J)
```

Listing 2: Full scan. Permanent-zone bookkeeping is inside extraction.

```
extract:
  flood 4-connected from the border through free or unknown
  mark unreached unknown as occupied in a copy
  record components of size >= 3 as permanent cell keys
  frontier cells = unknown cells with an 8-connected free neighbour
  drop the outer row and column of that mask
  return 8-connected clusters

score(clusters, D):
  for each cluster:
    reject if size < s_min, or no free target,
      or cell index within m of the border,
      or blacklisted, or distance outside [d_min, D],
      or openness < 0.15
    keep J from equation (score)
  if all kept distances > 0.35 D:
    recompute J with the far-frontier beta
  queue = up to 5 diverse large runners-up
  return the best
```

7 Blacklists

7.1 Ordinary entries

`_blacklist_goal` (lines 1140–1179) merges a new point into an existing key when the squared distance is at most $(1.5r_b)^2$, i.e. 1.2 m at the launched radius. Otherwise it inserts a new key with one hit and timeout $T_0 = 300$ s. On a merge the hit count increments and, if the count is now at least 2,

$$T = \min\left(T_0 \cdot 2^{h-1},\ 2400\right)\ \text{s}, \qquad \text{radius} = 2r_b. \tag{10}$$

The radius used at test time is r_b for one hit and $2r_b$ for any larger hit (lines 1222–1230). It does not grow with further hits. The timeout does, until the cap. The last blacklisted point is stored for the queue's $2r_b$ exclusion.

Expiry is checked at the start of an idle tick, not during a goal (lines 1241–1253). A goal can still be in the queue, and be popped, against an entry that has logically expired, until the next idle prune. Pop checks `_is_blacklisted`, which does not look at time. So expiry is idle-only.

7.2 Aborts and phantom successes

On `STATUS_ABORTED` the result callback blacklists and then, if the hit count stored under the exact returned position is at least 2, forces that counter to 10 and that timeout to 600 s (lines 621–644). The force uses the position Nav2 returned, not necessarily the merged key. If the merge wrote a different key, the force can miss it, while `_blacklist_goal` has already doubled the radius and scaled the timeout on the merged key. The log line can say "1st time" in that case. The exclusion is still the doubled one. This is source behaviour, not a guess about Nav2.

On `STATUS_SUCCEEDED`, residual unknown within 0.8 m increments a phantom counter whose key is the nearest existing phantom key inside 1.2 m, or the current point (lines 581–610). At 2, the code temporarily sets the base timeout to 600 s, calls `_blacklist_goal` on the current point (not necessarily the phantom key), restores the base timeout, and assigns hit count 10 on the current point. If the blacklist call merged into an older key, the scaled timeout uses 600 s as its base, so the second hit stores $\min(600 \cdot 2,\ 2400) = 1200$ s, not the 600 s named in the log. The log always prints "10 min". The stored duration can be 1200 s. The June report's threshold of 3 matches `6b7e6c2` and not the current source.

A success that does clear the unknown cells drops the phantom count for that exact key and clears the last-cancel exclusion.

7.3 Manual zones

`/manual_blacklist` accepts JSON (lines 1181–1205). `action: clear` deletes the manual list only. An add requires x and y. The radius defaults to r_b and is at least 0.1 m. `duration` ≤ 0 does not expire. The dashboard (`robot_dashboard_node.py`, lines 581–609) converts a click from `odom` into `map` with the current transform, and it clamps the radius to $[0.1, 5.0]$ m before publish. If the transform lookup fails, the translation used is zero and the click is treated as already in `map`.

Manual zones are tested first in `_is_blacklisted`. They are disks. They do not interact with hit counts.

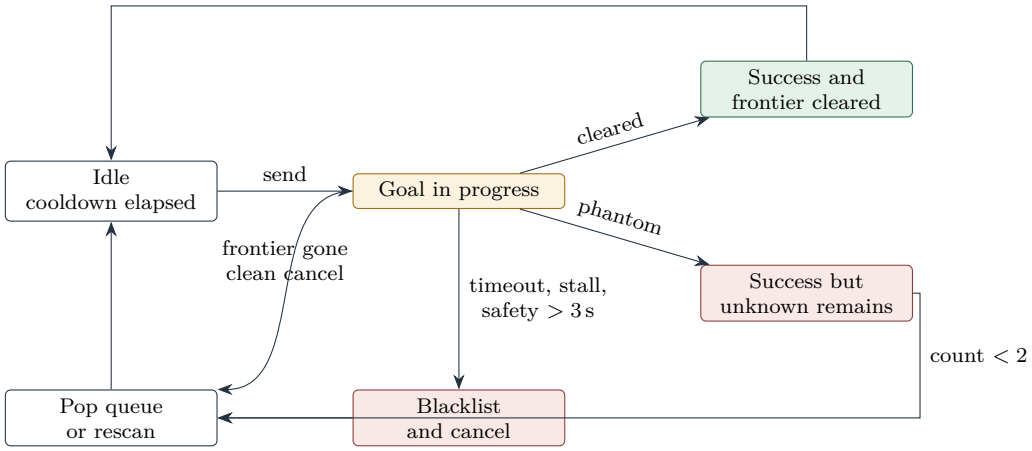


Figure 8: States that exist in the variables `goal_in_progress`, the action status, and the blacklist. There is no done state.

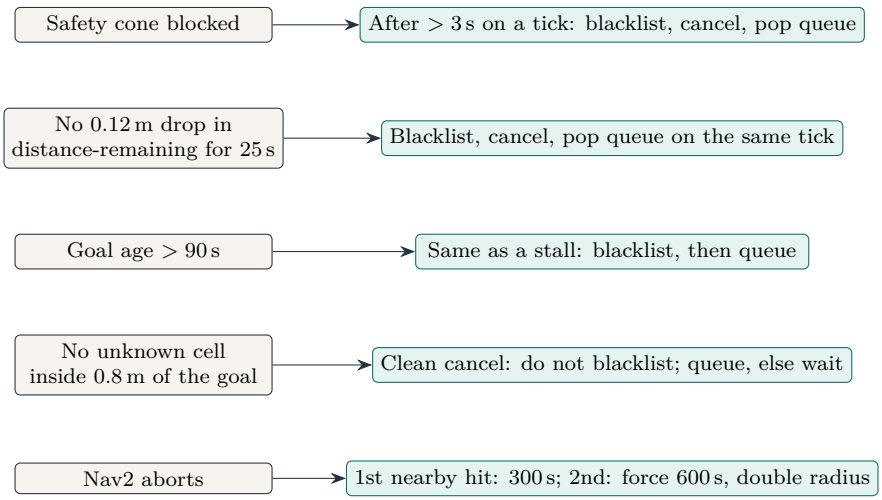


Figure 9: Events that cancel the active goal, and whether they blacklist. Sampled on the 3 s timer.

7.4 State machine

Table 4 is the authoritative event list. “Next” is what the source attempts on that path, not what a later idle tick might do if the attempt fails.

The abort row is easy to misread against the safety row. Safety and stall preempt inside the tick and can send a replacement before the result callback. The abort callback only blacklists. It does not pop. The next goal waits for cooldown and the idle path.

Table 4: Goal events. Radius and timeout assume a first hit at the launched r_b and T_0, and assume the merge key matches the queried point.

Event	Exclusion	Next action
Success, frontier cleared	None	Idle after cooldown
Success, unknown remains, count = 1	None	Idle after cooldown
Success, count reaches 2	Double radius; timeout uses a 600 s base	Idle after cooldown
Abort, first stored hit	0.80 m, 300 s	Idle; queue already not used inside the callback
Abort, second hit on that key	Double radius; timeout forced to 600 s	Same
Safety > 3 s with a queued point	Ordinary blacklist, then cancel	Send popped goal immediately
Stall or 90 s age	Ordinary blacklist, then cancel	Pop if possible, else next idle tick
Frontier vanished	None (clean cancel)	Pop if possible, else cooldown
Cancel status in the callback	None	Whatever the pre-cancel path already did
Goal rejected by Nav2	None	Cooldown, clear active goal

8 Calibration and relaxation

The first map callback calls `_calibrate_from_map` once (lines 178–212). Let L be the larger side of that grid in metres.

$$\begin{aligned} d_{\max} &\leftarrow \max\left(d_{\max},\, \min(0.5L,\, 12)\right), \\ N_{\text{done}} &\leftarrow \max\left(N_{\text{done}},\, \lfloor 2L \rfloor\right), \\ n_L &\leftarrow (7). \end{aligned} \tag{11}$$

$d_{\max}$ is only raised. A first grid whose half-diagonal-side is below 5.5 m leaves the search radius at the YAML value. L is the canvas, not the explored room and not the building. SLAM Toolbox's first published size is not fixed in the parameter file, so the numeric outcome of (11) is not determined by this repository. The formula is.

A 10 m canvas gives $0.5L = 5.0 < 5.5$, so the radius stays 5.5 m, and $\lfloor 2L \rfloor = 20$, so patience stays 20 ticks. The examples in the YAML comment ("10 m room → 5.0 m range") describe the formula's intermediate value, not the assignment, which refuses to lower the radius. A 20 m canvas raises the radius to 10 m and patience to 40. A 400×400 canvas at 0.05 m is 20 m on a side and also sets $n_L = \lfloor 0.0015 \cdot 160000 \rfloor = 240$ cells.

Relaxation runs only on ticks that found no goal (lines 214–250). It is sticky.

From $s_{\min} = 3$ and $d_{\min} = 0.35$ m the running values are: tick 3, size 2; tick 6, size 1 and $d_{\max} = 12$ m; tick 9, $d_{\min} = 0.175$ m. Further ticks do not reduce the size below 1. Patience 20 then logs completion and does not stop the timer. Empty ticks are not the same as elapsed time. Cooldown ticks and in-progress ticks do not increment the counter. Twenty empty ticks are at least 60 s of idle decisions, and longer if goals and cooldowns intervene.

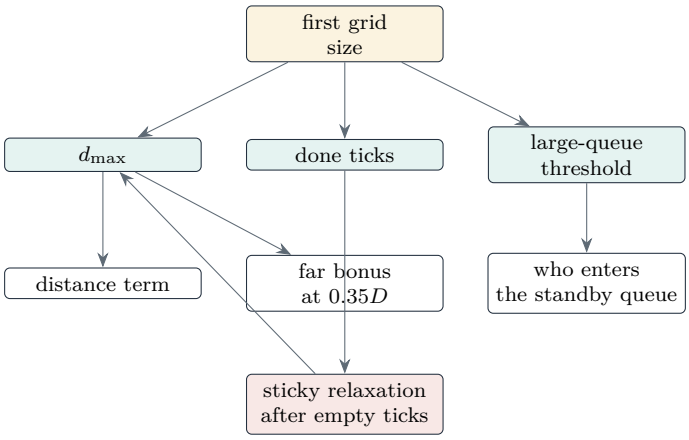


Figure 10: The first grid writes three knobs. Relaxation can later overwrite the search radius and the minimum distance. It does not rewrite n_L or the patience.

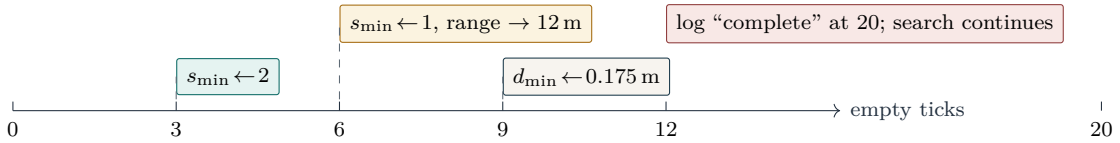


Figure 11: Schedule from the launched minimum size of 3. Tick 6 is the only moment at which the search radius is set to the cap by relaxation. Tick 9 halves the minimum distance once.

9 Worked arithmetic

None of these numbers was read from a robot log. They are evaluations of the formulas at the launched weights, $\rho = 0.05\,\mathrm{m}$, and $D = 5.5\,\mathrm{m}$.

Ordinary score. Cluster size 40, largest cluster 80, distance 2.2 m, heading error $\pi/2$, openness 0.60.

$$\sigma_{\text{size}} = 0.50, \qquad \sigma_{\text{dist}} = 2.2/5.5 = 0.40, \quad \sigma_{\text{head}} = 0.50, \quad \sigma_{\text{open}} = 0.60,$$
$$J = 1.8(0.50) - 1.0(0.40) - 0.5(0.50) + 0.7(0.60) = 0.67.$$

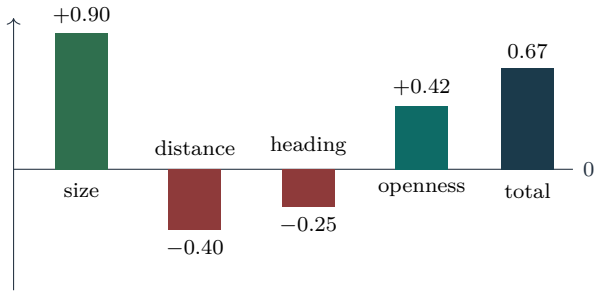
The four contributions are $+0.90$, -0.40, -0.25, $+0.42$.

A nearer, smaller cluster. Size 12, distance 0.80 m, heading error 0.20 rad, openness 0.40:

$$J = 1.8(12/80) - 0.80/5.5 - 0.5(0.20/\pi) + 0.7(0.40) \approx 0.373.$$

The larger cluster wins. The far bonus does not apply, because $0.80 < 0.35 \times 5.5 = 1.925$.

Far bonus. Both candidates at 4.0 m, heading error 0.5 rad, openness 0.70, largest size 80. For size 64, $\beta = \min(0.40, 0.45 \times 0.80) = 0.36$. For size 8, $\beta = 0.045$. Without the bonus the utilities are approximately 1.123 and -0.137. With it they are approximately 1.483 and -0.092. The large cluster gains 0.36 and the small one gains 0.045. Order does not change in this particular pair. A pair can be constructed where it does, because the bonus scales with size. That construction would still be arithmetic, not evidence that the bonus helps exploration.



Illustrative inputs, not a trial: $|F| = 40$, $|F|_{\max} = 80$, $d = 2.2\,\text{m}$, $D = 5.5\,\text{m}$, $|\Delta\psi| = \pi/2$, $\Omega = 0.60$.

Figure 12: Decomposition of the ordinary-score example. Illustrative inputs.

Pop of the ordinary candidate. If the same 2.2 m point is later popped, $J_{\text{pop}} = -2.2/5.5 = -0.40$, not 0.67. The idle gate against a stored 0.67 is $0.67 + 0.20 = 0.87$. The pop loses.

Border margin. $\text{int}(0.15/0.05) = \text{int}(2.999\ldots) = 2$. Cells 0 and 1 are rejected. Cell 2 is accepted. Its centre is 0.125 m from the map edge.

Rounding of permanent keys. For $\rho = 0.05$ and an origin on a grid line, the key of cell column i is $\text{round}(i + 0.5)$. Python half-to-even rounding gives the map $0 \to 0$, $1 \to 2$, $2 \to 2$, $3 \to 4$, $4 \to 4$. Columns 1 and 2 share a key. The mask is both coarser than one cell and not a simple dilation. Querying a goal at a cell centre uses the same formula, so a goal at the centre of a stored cell hits. A goal at the centre of an aliased neighbour can hit as well.

Timeout schedule. From a 300 s base, hit 1 stores 300 s at radius 0.80 m; hit 2 stores 600 s at 1.60 m; hit 3 stores 1200 s; hit 4 and after store 2400 s. The second-abort override then writes 600 s, which is shorter than a third-hit exponential timeout if that override runs late. Phantom insertion with the temporary 600 s base is a different sequence, as Section 8.2 describes.

Kinematic lower bounds. At 0.35 m/s with no rotation and no slowdown, 5.5 m takes 16 s and 12 m takes 34 s. At half speed inside PolygonSlow the 12 m bound is 69 s, still under the 90 s goal timeout if the whole path were slow. These are bounds on translation time. They are not travel times.

10 Parameters

Table 5 is the launched configuration. Table 6 is every hard-coded constant that changes a decision, a timeout, or a stored key. Marker sizes and colours are omitted; they do not affect the robot. Table 7 records node defaults that the launch file overrides, because a bare `ros2 run` would not reproduce the launched policy.

Table 5: Parameters loaded from `config/frontier_explorer.yaml` by `autonomous_robot.launch.py`.

Name	Value	Unit	Role
`map_topic`	`/map`	–	Occupancy grid
`map_frame`, `base_frame`	`map`, `base_footprint`	–	TF lookup
`goal_timeout_sec`	90	s	Age cancel
`goal_interval_sec`	3.0	s	Tick period
`goal_cooldown_sec`	3.0	s	Idle pause after a goal ends
`goal_reached_radius`	0.45	m	Clears a stored goal position when no goal is in progress
`progress_timeout_sec`	25	s	Stall window
`min_progress_delta`	0.12	m	Required drop in distance remaining
`min_frontier_size`	3	cell	Initial cluster floor
`min_goal_distance`	0.35	m	Near cutoff
`max_goal_distance`	5.5	m	Initial search radius
`max_range_expansion_factor`	2.0	–	One-shot rescore radius factor
`max_goal_distance_cap`	12	m	Ceiling for calibration, expansion, relaxation, and pops
`wall_safe_distance`	0.15	m	Input to (2); effective index margin is 2 cells
`goal_blacklist_radius`	0.80	m	Base exclusion; also scales merge, spread, and pop separation
`blacklist_timeout_sec`	300	s	Base timeout T_0
`size_weight`	1.8	–	w_s
`distance_weight`	1.0	–	w_d
`heading_weight`	0.5	–	w_h
`openness_weight`	0.7	–	w_o
`improvement_margin`	0.20	score	Idle hysteresis
`no_frontier_done_ticks`	20	tick	Patience before the completion log

Table 6: Hard-coded decision constants in `frontier_explorer_node.py`.

Symbol or site	Value	Effect
`_PHANTOM_THRESHOLD`	2 arrivals	Blacklist after repeated uncleared success
`_PHANTOM_TIMEOUT`	600 s	Forced abort timeout; temporary phantom base
`_queue_size`	5	Standby length
`_queue_max_age`	20 s	Discard and refresh
`_SAFETY_SWITCH_SEC`	3 s	Preempt threshold, sampled on the tick
Vanish and phantom radius	0.8 m	Unknown-cell neighbourhood; not r_b
Openness radius	3 cells	Window is 7×7

Symbol or site	Value	Effect
Openness floor	0.15	Hard reject
Far-zone fraction	0.35	Bonus only if all survivors are farther
Far bonus	$\min(0.40, 0.45\|S_f\|/\|S\|)$	Subtracted inside the distance term
Queue separation	$2.5r_b$	2.0 m when $r_b = 0.80$
Pop exclusion from last cancel	$2r_b$	1.6 m
Blacklist merge	$1.5r_b$	1.2 m
Repeat radius	$2r_b$	Applied when hits > 1
Timeout growth	$T_0 2^{h-1}$, cap 2400 s	On merged repeats
Forced hit count	10	Second abort and phantom; radius already saturated at 2
Permanent component minimum	3 cells	Smaller voids are filled but not stored
Permanent buffer	1 cell	Chebyshev
Permanent marker radius	$2r_b$	Display only
Relaxation period	every 3 empty ticks	Size, then range, then minimum distance
Size floor	1 cell	Relaxation stops
Minimum-distance floor	0.15 m	Half of 0.35 is 0.175, so the floor does not bind
Range-relax tick	6	Sets $d_{\max}$ to the cap
Distance-relax tick	9	Halves $d_{\min}$ once
Calibration factors	$0.5L$, $2L$, $0.0015WH$	First grid only
Default ρ before the first map	0.05 m	Overwritten on calibrate
Accept-server wait	1.0 s	Runs before any goal state is written. If the server is absent the tick returns and no goal is marked in progress.
Manual radius floor	0.1 m	Explorer side; dashboard also caps at 5 m

11 Complexity

A full scan allocates boolean masks of size WH and runs two BFS passes in the worst case: the border fill, and a labelling of the enclosed set. Frontier clustering is linear in the number of frontier cells, with 8-neighbour lookups in a hash set. Scoring is linear in the number of clusters times a 7×7 window. The far-bonus rescore repeats the openness window. Permanent-zone insertion writes nine keys per enclosed cell and is not repeated for keys already present, but the flood fill itself is repeated on every scan.

The idle period is 3 s. During a goal, a scan runs when the queue is older than 20 s. No timing

Table 7: Node defaults that differ from the YAML. Irrelevant if the launch file is used. Dangerous if it is not.

Parameter	Declare default	YAML
`min_frontier_size`	8	3
`min_goal_distance`	0.40 m	0.35 m
`max_goal_distance`	6.0 m	5.5 m
`goal_blacklist_radius`	0.50 m	0.80 m
`blacklist_timeout_sec`	90 s	300 s
`size_weight`	1.5	1.8
`heading_weight`	0.8	0.5
`openness_weight`	0.4	0.7
`improvement_margin`	0.25	0.20
`goal_cooldown_sec`	2.0 s	3.0 s
`progress_timeout_sec`	18 s	25 s
`min_progress_delta`	0.15 m	0.12 m
`no_frontier_done_ticks`	4	20
`wall_safe_distance`	0.50 m	0.15 m

measurement is in the repository. The July notes about a Raspberry Pi 5 missing controller cycles refer to SLAM scan matching on DenseBoost clouds of about 3200 returns, not to this Python loop. Whether the flood fill is negligible beside that load is a **gap**.

Memory for permanent keys grows with the enclosed area and is never released. On a long run in a building with many small voids this set can become large. There is no measurement of its size.

12 History

Figure 13 is the decision history. Table 8 is the same history with the behaviour that actually changed. Hardware-only commits are omitted here; the July navigation commits are included only where they change a signal the explorer reads or a fate of the goals it sends. Line counts are `wc -l` on `frontier_explorer_node.py` at that commit.

Table 8: Behavioural history. "Record" is the commit message or the comment added with it. Where that text does not state a reason, the cell says so.

Date	Hash	Lines	What changed, and the record
12 May	`f493c96`	517	Restored after a deleted branch. Vectorised 8-connected extraction, utility, expansion, openness floor 0.25, border 0.60 m, blacklist 0.50 m / 90 s, progress window 35 s, patience 10. No queue, flood fill, or phantom counter.
13 May	`82dbf87`	514	Removed the 0.25 floor. Record: it rejected all tight frontiers in warehouse and office worlds.
13 May	`48d893c`	589	First-map calibration and relaxation. Record is the title. No before/after numbers.

Date	Hash	Lines	What changed, and the record
14 May	`9a3fee8`	–	Message: tune inflation, SLAM rate, and blacklist for hardware. YAML continues to move; the values that remain are those of 24 May.
24 May	`76eae0d`	–	Border 0.60 → 0.15 m. Progress window 35 → 25 s. Comment: 0.60 m removed every wall-adjacent frontier because the walls lay on the map boundary.
24 May	`d81dd84`	599	Stuck-position handling, faster relaxation, patience 20, cap 12 m, blacklist 0.80 m / 300 s. This is the last commit that edits the YAML.
2 Jun	`3c81dd1`	–	Cancel when the frontier is mapped during the trip.
2 Jun	`827026d`	766	Queue and safety preemption. Record: "zero standstill time", as an intention.
2 Jun	`14edb83`	–	Repeat failures double the radius. Minimum distance from the stuck point.
3 Jun	`6b7e6c2`	890	Phantom counter, threshold 3, timeout 600 s. Openness floor returns at 0.15.
3 Jun	`b75e03f`	907	Every abort treated as a long dead zone. Superseded the same day.
3 Jun	`f1a9bd3`	984	Enclosed fill and a diverse queue.
3 Jun	`b8676df`	1030	Queue prefers large clusters. Far bonus.
3 Jun	`ab82950`	1144	Markers. Exponential timeout.
3 Jun	`818e294`	1244	Idle path uses the queue first. Permanent zones gain a long life. Markers kept visible.
3 Jun	`276f5cb`	1254	First abort is an ordinary blacklist. A second abort at that area is the long exclusion. Record: costmap resize can abort a reachable goal.
3 Jun	`29348bf`	–	Report v2, on the simulation history. Describes threshold 3 and a circular permanent zone.
10 Jun	`b79ad0d`	1291	Threshold 3 → 2. n_L gains the $0.0015WH$ term. Vanish and timeout paths pop the queue.
10 Jun	`ee00c4d`	1318	Clean-cancel no longer blacklists. Queue refreshes during a goal.
10 Jun	`47a164d`	1322	Cancel callback does not blacklist. Removes a double write.
10 Jun	`badd582`	1341	Generation counter so a stale callback cannot clear the new goal.
10 Jun	`6ed4bb4`	–	Nearby permanent centroids merge. Record: centroid drift duplicated markers.
10 Jun	`5bbaf0f`	1371	Permanent test becomes the cell set. Record: replaces the 1.6 m circle.

Date	Hash	Lines	What changed, and the record
19 Jul	`eccbb97`	1448	Manual zones from the dashboard. Last edit of the explorer.
19 Jul	`e48f411`, `19bbea1`	–	Rotate-to-heading off, then on again once the safety stop halts on dead `/odom_raw`.
26 Jul	`91d464f`, `8547161`	–	Cone narrowed to 50° with hysteresis. PolygonSlow enabled. Explorer source unchanged.

Branch `home_updated` adds a home goal after `home_recovery_ticks` (default 6) and is not merged. The record of why `main` lacks it is absent. The June report's branch table, which describes `main` as a 599-line explorer without a queue, describes an older `main`. Current `main` contains the June–July policy.

13 Where the June report is wrong

The note `docs/frontier_explorer_report_v2.md` at `29348bf` (3 June 2026) is a useful diary and a bad specification of the current source. Table 9 lists disagreements that change a technical claim. Wording differences are ignored.

Table 9: Legacy report against the source on `main`.

Topic	Report v2	Current source
Phantom threshold	3	2, since `b79ad0d`
Permanent exclusion	Disk of radius 1.60 m	Exact buffered cells; disk is only the marker
Large-queue rule	$\max(3s_{\min}, 15)$	Also $\lfloor 0.0015WH \rfloor$
Vanish during a trip	Rescan next tick	Pop the queue first
Goal timeout	Rescan next tick, in the summary table	Pop the queue on that tick
Termination	Node stops sending	Log only
Safety geometry	0.25 m, front 180°, rear 60°	0.35 m, both arcs 50°, hysteresis, odometry watchdog
Safety topic rate	20 Hz	On each scan, plus the 10 Hz watchdog when odometry is dead
RPP collision check	Off, and described as the only laser gate	Still off, but PolygonSlow is now in front of the safety stop
Border margin	3 cells	$\text{int}(0.15/0.05) = 2$ cells
Completion formula	10 m room → 5 m range	Half of 10 m is below 5.5 m, so the radius is not lowered

The report's scoring equation, the 0.15 openness floor, the far-bonus formula, the 3 s safety switch, the exponential timeout, and the two-stage abort match the source at the launched parameters. Those parts can be reused. The rest should not be quoted as current behaviour.

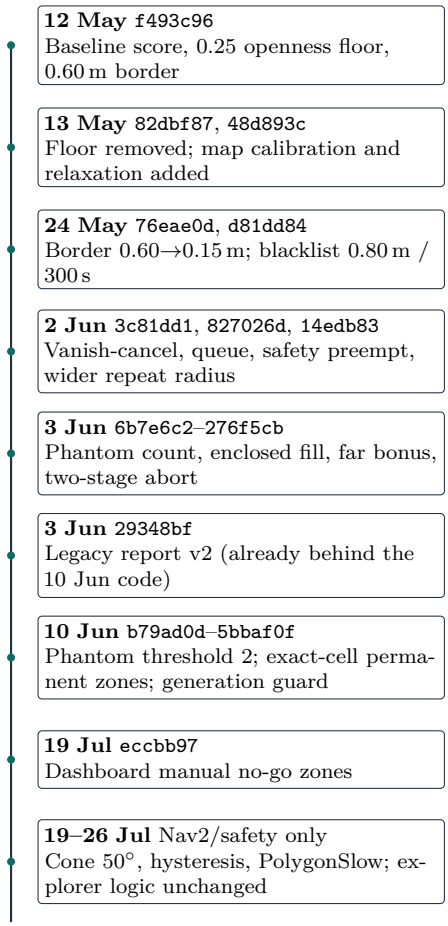


Figure 13: Commits that changed exploration behaviour or the document that claims to describe it. Dates are the commit dates.

14 Evidence

Searches of the tree found no rosbag, no CSV of runs, and no coverage log. The word "verified" in July commit messages attaches to transform tolerance, a laser-yaw correction, and the safety cone. Those messages do not state a sample size, a baseline, or an exploration metric. This report does not promote them to results.

Supported now:

- the decision rule, the constants, and the control flow in the sections above;
- the existence of the design reversals in Section 12, as text in commits;
- the disagreements in Table 9.

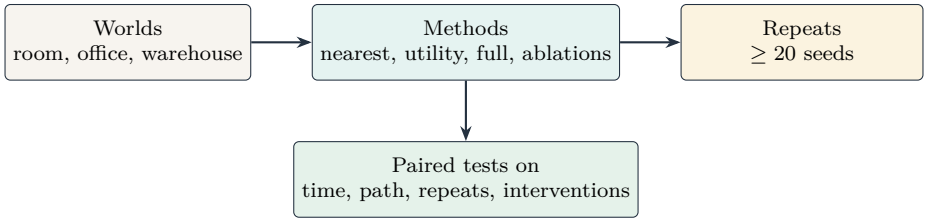
Not supported:

- any comparison of time, path, or coverage against another explorer;
- any ablation of the flood fill, the phantom counter, the queue, the far bonus, or relaxation;
- the claim that standstill time fell to zero;
- the claim that two phantom hits are better than three;

- the claim that exploration completes.

Negative findings that *are* supported, because they are properties of the source rather than of a trial: the completion log does not stop the node; the pop score is not the selection score; the border comment does not match (2); the permanent marker is larger than the permanent predicate; the first grid can set a queue threshold that the fallback then undoes.

15 What an evaluation has to be



Not yet run. No chart in this paper uses measured exploration outcomes.

Figure 14: Protocol. Empty of results on purpose.

Systems. The same robot model, the same Nav2 file, and the same SLAM file for every method. Methods: nearest frontier with the same detector and the same blacklist-free pruning; a gain-over-cost utility whose cost is Nav2 path length and whose gain is cluster size, which is the honest `explore_lite`-style baseline using this detector; the full policy; and ablations that remove exactly one of enclosed fill, phantom counter, queue preemption, far bonus, and relaxation. A path-cost version of the full policy is a separate row, because Euclidean cost is a confound.

Worlds. The office and warehouse worlds named in `82dbf87` if they can be recovered from the `Simulation` history, plus one hardware floor with a saved map and a written layout. Worlds are not on `main` today. A comparison that uses a different map for each method is not a comparison.

Repeats and outcomes. At least twenty starts per cell, with the start pose fixed across methods and the simulator seed varied. Outcomes: time and path length to 95 % of the reachable free space that a human labels in advance; number of goals; number of goals whose point lies within 1.2 m of an earlier goal; safety-preemption count; abort count; phantom count; fraction of ticks that pop the queue instead of scanning. The reachable set has to be defined without the explorer's own flood fill, or the enclosed-region ablation cannot be scored fairly. One workable definition is the free space observed by a reference run that is not part of the comparison, unioned across methods after the fact and frozen before scoring.

Statistics. Paired tests within a world, because the start is shared. Report medians and interquartile ranges, not only means. A single impressive run is not a result. Stopping rules and excluded crashes are written down before the runs.

Instrumentation that the code does not yet have. The debug line inside scoring is at debug level and is not a trial log. A later experiment needs a structured record per goal: stamp, pose, J

and J_{pop} if both exist, cluster size, reason for ending, whether the queue was used, blacklist hits, and the unknown-cell count within 0.8 m. Without that record the phantom counter cannot be audited after the fact.

Until this protocol is run, the correct sentence in a conference abstract is that the policy is implemented and uncompared.

16 Threats to validity

1. Euclidean cost favours frontiers behind walls. A gain that looks helpful may be an artefact of that cost.
2. Cluster size is not area that the lidar will see. Openness is a 7×7 free fraction, not clearance along the path.
3. The 3 s sampler delays every preemption by up to one period and defines "25 s" as "the first tick after 25 s".
4. One-shot calibration on the SLAM canvas makes thresholds depend on a middleware detail that this repository does not pin down.
5. Key rounding aliases permanent cells. The RViz disk hides the alias.
6. Queue-first idle selection serves stale runners-up for up to 20 s.
7. J_{pop} makes the hysteresis test and the log misleading whenever a stored four-term score is still present.
8. The safety Boolean conflates a forward obstacle with a dead encoder.
9. Equal 25 s windows on different progress definitions let the explorer blacklist a robot that Nav2 considers to be progressing, and the reverse can occur when remaining distance falls during a spin that does not change pose enough for Nav2.
10. A null `goal_handle` during early preemption is an unhandled exception path.
11. Semantic perception is unused, so the method neither benefits from nor is confounded by the detector. It also cannot be sold as semantic exploration.
12. The author of this report did not run the stack. A defect that is invisible statically, such as a QoS mismatch on `/map`, is not ruled out. The map subscription uses the default QoS. SLAM Toolbox map publishers are often transient-local. A default volatile subscription can miss the latched map. This report flags it as a risk, not as a reproduced failure. It should be checked before any trial.

The QoS risk is specific enough to state carefully. The subscription is created as `create_subscription(..., 10)` (line 155), which is a depth-10 volatile reliable subscription in rclpy. If the map publisher is transient-local, late start at 22 s can still receive data only when the publisher sends the next update. SLAM Toolbox's `map_update_frequency` is 2.0 Hz, so a later update should arrive. A truly latched and silent map would be missed. Whether that happens on this version of SLAM Toolbox was not tested here.

17 Reproducibility

The following reproduces the *configuration*, not a measured outcome.

1. Revision `de8209a` of the public repository `amey1942007/Ros_lidar_bot`. The explorer bytes match `eccbb97`.
2. ROS 2 Jazzy, the Nav2 bringup that `launch_robot.launch.py` includes, and SLAM Toolbox's `online_async_launch.py`. Versions are not pinned in the repository beyond the Jazzy migration commits of 10–16 July 2026.
3. `ros2 launch Ros_lidar_bot autonomous_robot.launch.py` on the robot described by the URDF and the launch file: differential drive, footprint $0.50 \times 0.36\,\mathrm{m}$ in the costmap, lidar yaw π in `lidar.xacro`, scan mode DenseBoost. Motor and lidar product names appear in comments (DDSM115, RPLidar S2E). This report does not repeat them as independently measured hardware.
4. The YAML in Table 5 and the hard-coded constants in Table 6. Changing either table changes the policy.
5. Worlds for the May comments are not on `main`. Recover them from the `Simulation` history if a simulation repeat is required. Do not invent a substitute and call it the same experiment.

A second machine that runs the node without the launch file will silently take Table 7. Any replication protocol has to assert that the YAML was loaded, for example by reading the parameters back from the node.

Logging at INFO shows goal points, phantom counts, aborts, and the completion line. It does not show the four terms of J. Those are debug logs inside scoring. A replication that needs them must enable debug for this node or add the structured log in Section 15.

18 Publication and intellectual property, in brief

The conference argument is in the concise manuscript. The short version of the judgment, justified at length in the separate assessment, is as follows.

The work is not ready for IROS, ICRA, or RA-L as a results paper. The method is describable, and this report describes it. The missing piece is the comparison in Section 15. A workshop paper that presents the policy as an engineered response to named failure modes, with the gaps left visible, is a more honest near-term target than a claim of improved exploration.

The repository is public and licensed Apache-2.0. That is a disclosure. It is also a patent grant to users of the code, under the terms of that licence. Whether any residual claim is worth filing is a question for counsel, and it has to be asked before a conference version is posted and with the disclosure date of the May 2026 commits in hand. This report is not a patent application and not legal advice.

19 Conclusion

The explorer on `main` detects classical frontiers on a full grid, scores them with (5), and wraps that score in a failure policy: enclosed unknown cells are removed and stored as rounded world keys;

an uncleared Nav2 success is counted and blacklisted at two; repeated aborts widen and lengthen an exclusion; a safety Boolean, a stall, and a timeout can jump to a spatially separated runner-up. Relaxation loosens size, range, and minimum distance, and then keeps them loose. The node does not stop when it says exploration is complete.

Several of those mechanisms exist because earlier thresholds made the robot refuse real frontiers or stand still. The commits say so. They do not show that the replacements explore better than nearest-frontier or `explore_lite`. The June report describes an intermediate program and should not be cited as the specification. The specification is the source, with the line ranges and tables in this report, until the code changes.

The useful next edit is small and prior to any new heuristic: make a popped goal carry J rather than (8), make the border test a distance, stop aliasing permanent keys, and latch or delete the completion log. The useful next measurement is the protocol in Section 15. Either of those is more informative than another weight.

References

[1] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proc. IEEE Int. Symp. Computational Intelligence in Robotics and Automation*, 1997, pp. 146–151.

[2] B. Yamauchi, "Frontier-based exploration using multiple robots," in *Proc. Int. Conf. Autonomous Agents*, 1998, pp. 47–53.

[3] H. H. González-Baños and J.-C. Latombe, "Navigation strategies for exploring indoor environments," *Int. J. Robotics Research*, vol. 21, no. 10–11, pp. 829–848, 2002.

[4] A. A. Makarenko, S. B. Williams, F. Bourgault, and H. F. Durrant-Whyte, "An experiment in integrated exploration," in *Proc. IEEE/RSJ IROS*, 2002.

[5] C. Stachniss, G. Grisetti, and W. Burgard, "Information gain-based exploration using Rao-Blackwellized particle filters," in *Proc. Robotics: Science and Systems*, 2005.

[6] D. Holz, N. Basilico, F. Amigoni, and S. Behnke, "Evaluating the efficiency of frontier-based exploration strategies," in *Proc. ISR/ROBOTIK*, 2010.

[7] M. Keidar and G. A. Kaminka, "Efficient frontier detection for robot exploration," *Int. J. Robotics Research*, vol. 33, no. 2, pp. 215–236, 2014.

[8] P. Quin, A. Alempijevic, G. Paul, and D. Liu, "Expanding wavefront frontier detection: an approach for efficiently detecting frontier cells," in *Proc. Australasian Conf. Robotics and Automation*, 2014.

[9] H. Umari and S. Mukhopadhyay, "Autonomous robotic exploration based on multiple rapidly-exploring randomized trees," in *Proc. IEEE/RSJ IROS*, 2017.

[10] J. Hörner, "Map-merging for multi-robot system," Bachelor's thesis, Charles University, Prague, 2016.

[11] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ IROS*, 2020.

[12] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *J. Open Source Software*, vol. 6, no. 61, art. 2783, 2021.

[13] S. Macenski, S. Singh, F. Martín, and J. Ginés, "Regulated pure pursuit for robot path tracking," *Autonomous Robots*, vol. 47, pp. 685–694, 2023.